

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

«МИРЭА - Российский технологический университет
Институт перспективных технологий и индустриального программирования
Кафедра индустриального программирования»

ПРАКТИЧЕСКОЕ ЗАНЯТИЕ №2

Тема: «Выбор темы проекта. Сбор требований. Написание документации
проекта (SRS) и UML-диаграммы»

Проект: «Симулятор работы лифтов в здании»

Выполнила:
студентка группы ЭФБО-06-24
Кальченко Евгения Витальевна

Преподаватель:
Сокунов Дмитрий Антонович

Москва, 2026

Введение

Данный документ представляет собой техническое задание (Software Requirements Specification, SRS), целью которого является определение архитектуры и детализация требований к разрабатываемой корпоративной системе «Симулятор работы лифтов в здании». Основная задача создаваемой системы — многопоточное моделирование работы нескольких пассажирских лифтов, обрабатывающих вызовы с различных этажей здания, и сбор статистики обслуживания этих вызовов.

Область действия проекта охватывает создание консольной программы для имитации работы группы лифтов. Система выполняет приём вызовов от пассажиров, диспетчеризацию заявок между лифтами по выбранной стратегии, перемещение лифтов между этажами и формирование итогового журнала событий и метрик. Стоит отметить, что в рамках текущей учебной реализации работа с реальным оборудованием (физическими лифтами или контроллерами) не предусмотрена — все процессы моделируются программно с использованием стандартных средств многопоточности языка C++.

Общее описание программного продукта

Программный комплекс выступает учебной моделью работы группы лифтов в многоэтажном здании. Проект реализуется на C++ без тяжеловесных фреймворков для демонстрации работы с механизмами многопоточности, синхронизации и событийного моделирования.

Основные функции системы:

- Приём вызовов с этажей и их постановка в очередь диспетчеризации.
- Распределение заявок между лифтами по выбранной стратегии диспетчеризации.
- Параллельное перемещение нескольких лифтов между этажами с учётом их текущих очередей.
- Ведение журнала событий в консоли с указанием времени наступления и идентификатора лифта.
- Сбор итоговой статистики: количество обработанных заявок, среднее и максимальное время ожидания, загрузка каждого лифта.

Единственным прямым пользователем выступает пользователь-оператор, запускающий симуляцию и задающий её параметры. Реальные пассажиры и физические лифты в системе не участвуют — их поведение моделируется внутренним генератором событий.

Спецификация требований (Детальные требования)

В процессе сбора требований к будущей учебной системе применялись традиционные методологии системного анализа. Основной упор был сделан на изучение принципов

диспетчеризации в многолифтовых системах и моделей конкурентного программирования. Дополнительно применялся метод моделирования интервью с потенциальным заказчиком в лице преподавателя для выявления ключевых учебных целей (многопоточность, синхронизация, объектно-ориентированное проектирование).

С точки зрения учебных целей, внедрение решения должно продемонстрировать корректную работу нескольких параллельных лифтов под нагрузкой не менее 100 одновременных заявок без потерь и блокировок. Это позволит оценить эффективность выбранной стратегии диспетчеризации и качества синхронизации общего состояния системы.

Пользовательские требования формировались на основе гибкого подхода User Stories. Пользователю-оператору необходима возможность задавать число этажей, число лифтов и длительность симуляции через параметры запуска или конфигурационный файл. Также требуется вывод логов в консоль с указанием времени события, идентификатора лифта, текущего этажа и статуса заявки для мониторинга работы системы. По завершении симуляции должна выводиться сводная статистика по каждому лифту и системе в целом.

Функциональные требования:

1. Приём вызовов с этажей с указанием направления (вверх/вниз).
2. Назначение каждого вызова конкретному лифту по выбранной стратегии диспетчеризации.
3. Параллельное движение лифтов между этажами по собственным очередям целей.
4. Обработка заявки внутри кабины (выбор целевого этажа после посадки пассажира).
5. Логирование ключевых событий: приём заявки, прибытие на этаж, открытие/закрытие дверей.
6. Сбор и вывод итоговой статистики: число обработанных заявок, среднее и максимальное время ожидания, загрузка каждого лифта.

Нефункциональные требования (ограничения):

7. **Язык реализации:** C++17 и выше (поточная логика реализуется средствами стандартной библиотеки).
8. **Конкурентность:** не менее одного потока на каждый лифт плюс поток диспетчера/генератора заявок.
9. **Потокобезопасность:** доступ к общим очередям и состояниям лифтов защищён примитивами синхронизации без активного ожидания (busy waiting).
10. **Производительность:** время диспетчеризации одной заявки — не более 10 мс при числе лифтов до 10 и числе этажей до 50.
11. **Надёжность:** программа не должна аварийно завершаться при пиковом поступлении вызовов и должна корректно обрабатывать сигнал завершения симуляции.

Визуальное моделирование (Диаграммы UML)

В данном разделе представлены описания UML-моделей, иллюстрирующих архитектуру и поведенческие особенности разрабатываемой системы.

12. **Диаграмма вариантов использования (Use Case):** демонстрирует роли.

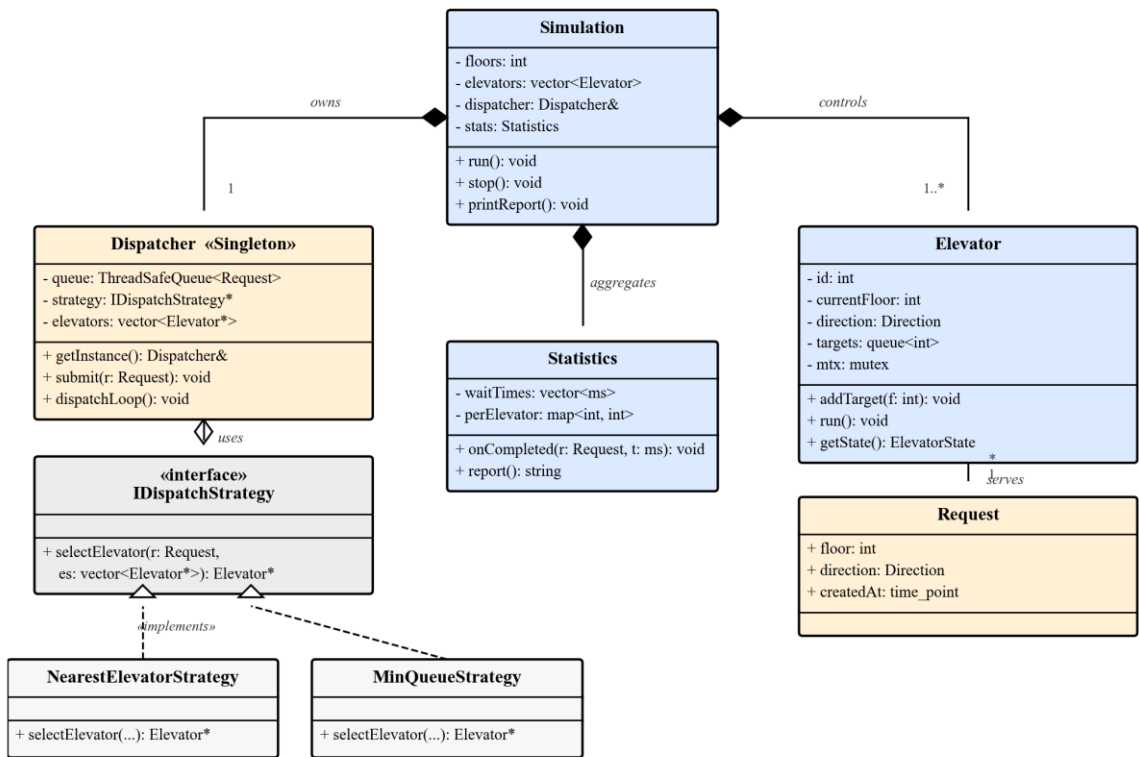
Пользователь-оператор инициирует запуск симуляции, задаёт параметры и наблюдает итоговый журнал. Внутренний генератор заявок и диспетчер показаны как включаемые сценарии «Принять вызов с этажа» и «Назначить лифт».



13. **Диаграмма классов (Class Diagram):** отражает статическую структуру. Класс

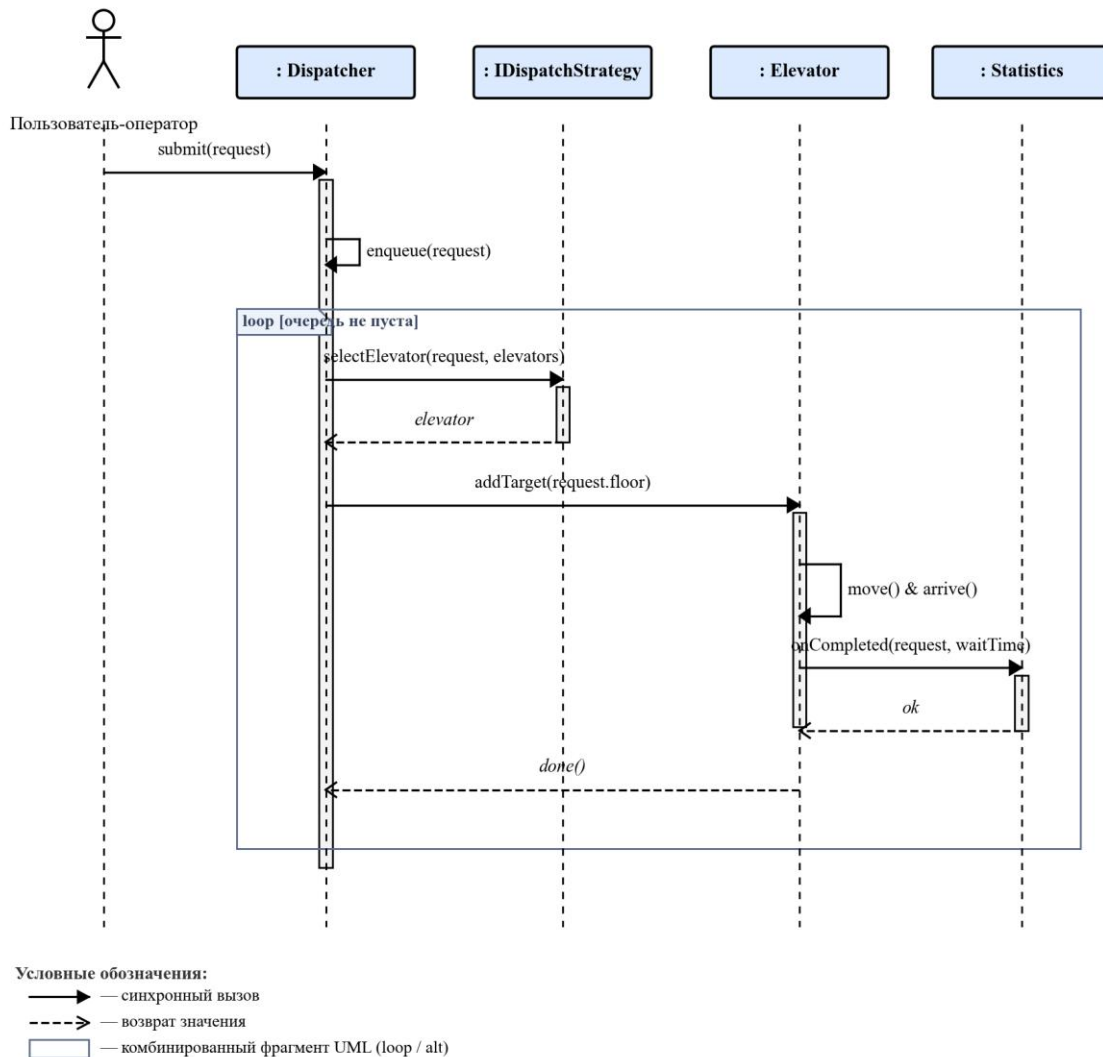
Simulation управляет жизненным циклом программы и связан с объектами Elevator (отдельные лифты) и Dispatcher (диспетчер заявок). Класс Request описывает заявку (этаж вызова, направление, временную метку). За сбор метрик отвечает Statistics, агрегирующий данные по обработанным заявкам и загрузке лифтов.

Диаграмма классов «Симулятор работы лифтов в здании»



14. **Диаграмма последовательности (Sequence Diagram):** иллюстрирует сценарий обработки вызова с этажа. Генератор событий создаёт Request → Dispatcher выбирает подходящий Elevator → Elevator перемещается к этажу вызова → фиксирует прибытие и принимает пассажира → завершает заявку и обновляет Statistics.

Диаграмма последовательности «Обработка вызова с этажа»



5. Паттерны проектирования

На основании требований к системе и спроектированной UML-архитектуры, для реализации учебного симулятора лифтов были выбраны следующие паттерны проектирования:

5.1. Порождающий паттерн: Singleton (Одиночка)

Обоснование: Системе необходима единая, глобально доступная точка управления заявками и распределением лифтов. Создание нескольких экземпляров диспетчера привело бы к несогласованному назначению заявок, дублированию обработки и нарушению целостности статистики обслуживания.

Применение в проекте: Класс Dispatcher будет реализован как Singleton.

- Конструктор класса объявляется приватным.
- Доступ к экземпляру осуществляется через статический метод `Dispatcher::getInstance()`.
- Внутри класса используется потокобезопасная блокировка (`std::mutex`) для безопасной постановки заявок в очередь и выборки подходящего лифта из нескольких рабочих потоков.

5.2. Поведенческий паттерн: Strategy (Стратегия)

Обоснование: В учебной системе может потребоваться сравнить разные стратегии назначения лифтов: ближайший свободный, минимальная очередь, лифт того же направления. Паттерн «Стратегия» позволит инкапсулировать эти алгоритмы и менять их без изменения ядра диспетчера.

Применение в проекте:

- Будет создан интерфейс `IDispatchStrategy` с методом `selectElevator(request, elevators)`.
- Создаются конкретные классы `NearestElevatorStrategy` и `MinQueueStrategy`, реализующие разные правила выбора.
- Класс `Dispatcher` будет принимать объект `IDispatchStrategy` через конфигурацию при запуске.

5.3. Архитектурный паттерн: Thread Pool / Producer-Consumer

Обоснование: В нефункциональных требованиях указана поддержка не менее 100 одновременных заявок. Создание нового потока на каждую заявку ресурсоёмко. Очередь заявок и набор рабочих потоков лифтов решают эту задачу за счёт переиспользования заранее созданных потоков.

Применение в проекте:

- Генератор событий выступает «производителем» и помещает `Request` в потокобезопасную очередь `Dispatcher`.
- Потоки лифтов выступают «потребителями»: каждый `Elevator` работает в собственном потоке и забирает из своей очереди целей следующее назначение.
- Синхронизация выполняется через `std::mutex` и `std::condition_variable`, чтобы избежать активного ожидания.

6. Детальное описание применения паттернов проектирования

В предыдущей главе паттерны проектирования были перечислены на уровне обоснования архитектурных решений. В данной главе для каждого выбранного паттерна приводится подробное описание его применения на конкретных классах системы и иллюстрируется фрагментами кода на C++17, отражающими предполагаемую реализацию.

6.1. Singleton: класс Dispatcher

Назначение в проекте. Класс Dispatcher выступает единой точкой управления потоком заявок: принимает Request от генератора событий, хранит стратегию назначения и общую очередь, доступную всем потокам лифтов. Любая попытка завести второй экземпляр привела бы к рассогласованию очереди и нарушению целостности статистики обслуживания.

Реализация. Конструктор объявлен приватным, копирование и присваивание запрещены. Единственный экземпляр инициализируется ленивым статическим объектом в методе getInstance(), что обеспечивает потокобезопасную инициализацию средствами C++11.

```
class Dispatcher {
public:
    static Dispatcher& getInstance() {
        static Dispatcher instance;
        return instance;
    }

    Dispatcher(const Dispatcher&) = delete;
    Dispatcher& operator=(const Dispatcher&) = delete;

    void submit(Request request);
    void dispatchLoop();
    void setStrategy(std::unique_ptr<IDispatchStrategy> s);

private:
    Dispatcher() = default;

    ThreadSafeQueue<Request> queue_;
    std::unique_ptr<IDispatchStrategy> strategy_;
    std::vector<Elevator*> elevators_;
};
```

Внутреннее состояние Dispatcher защищено потокобезопасной очередью queue_ и блокировкой полей, к которым обращаются потоки лифтов. Класс Simulation на старте получает ссылку Dispatcher::getInstance() и регистрирует в нём перечень лифтов, после чего может многократно вызывать submit() из любого потока.

6.2. Strategy: интерфейс IDispatchStrategy

Назначение в проекте. Алгоритм выбора лифта для заявки изолирован от Dispatcher: при разных нагрузках уместны разные стратегии (ближайший лифт, минимальная очередь, лифт того же направления). Паттерн позволяет менять алгоритм во время запуска без повторной сборки.

```
struct IDispatchStrategy {
    virtual ~IDispatchStrategy() = default;
    virtual Elevator* selectElevator(
        const Request& request,
        const std::vector<Elevator*>& elevators) = 0;
};

class NearestElevatorStrategy : public IDispatchStrategy {
public:
    Elevator* selectElevator(
        const Request& request,
        const std::vector<Elevator*>& elevators) override;
};

class MinQueueStrategy : public IDispatchStrategy {
public:
    Elevator* selectElevator(
        const Request& request,
        const std::vector<Elevator*>& elevators) override;
};
```

Dispatcher хранит указатель IDispatchStrategy* и вызывает selectElevator() при каждой итерации цикла диспетчеризации. Подмена стратегии выполняется через метод setStrategy() — например, по конфигурационному параметру dispatcher_strategy = "nearest" / "min_queue".

6.3. Factory Method: фабрика стратегий

Назначение в проекте. Конкретный тип стратегии определяется на основе строкового значения из конфигурации. Если оставить switch/if в Dispatcher, добавление каждой новой стратегии будет ломать клиентский код. Factory Method переносит выбор класса в одну точку расширения.

```
struct StrategyFactory {
    static std::unique_ptr<IDispatchStrategy>
    create(const std::string& name) {
        if (name == "nearest")
            return std::make_unique<NearestElevatorStrategy>();
        if (name == "min_queue")
            return std::make_unique<MinQueueStrategy>();
        throw std::invalid_argument(
            "Unknown dispatch strategy: " + name);
    }
};
```

Класс `Simulation` при инициализации вызывает `Dispatcher::getInstance().setStrategy(StrategyFactory::create(config.strategy))`, что позволяет добавлять новые стратегии без правок `Dispatcher` и `Simulation` — достаточно расширить одну функцию `create()`.

6.4. Producer-Consumer: класс `ThreadSafeQueue`

Назначение в проекте. Внешний генератор заявок и набор потоков-лифтов работают конкурентно. Чтобы исключить активное ожидание и потерю заявок, между ними ставится потокобезопасная очередь с блокирующим `pop()`.

```
template <typename T>
class ThreadSafeQueue {
public:
    void push(T value) {
        {
            std::lock_guard<std::mutex> lock(mtx_);
            queue_.push(std::move(value));
        }
        cv_.notify_one();
    }

    T waitAndPop() {
        std::unique_lock<std::mutex> lock(mtx_);
        cv_.wait(lock, [this] { return !queue_.empty() || done_; });
        T value = std::move(queue_.front());
        queue_.pop();
        return value;
    }

    void shutdown() {
        {
            std::lock_guard<std::mutex> lock(mtx_);
            done_ = true;
        }
        cv_.notify_all();
    }

private:
    std::queue<T> queue_;
    std::mutex mtx_;
    std::condition_variable cv_;
    bool done_ = false;
};
```

Producer — это `Dispatcher::submit()`, вызываемый из потока генератора. Consumer — поток `Dispatcher::dispatchLoop()`, извлекающий заявки и распределяющий их по лифтам. Каждый `Elevator` также пользуется собственной маленькой очередью целей по той же схеме.

6.5. Observer: подписка `Statistics` на события лифта

Назначение в проекте. Лифт обязан сообщать о завершении обслуживания каждой заявки. Жёсткая зависимость Elevator от Statistics затруднит добавление новых потребителей событий (например, веб-интерфейса мониторинга). Observer развязывает источник событий и подписчиков.

```
struct IRequestCompletionListener {
    virtual ~IRequestCompletionListener() = default;
    virtual void onCompleted(
        const Request& request,
        std::chrono::milliseconds waitTime) = 0;
};

class Statistics : public IRequestCompletionListener {
public:
    void onCompleted(const Request& request,
        std::chrono::milliseconds wait) override;

    std::string report() const;
};

class Elevator {
public:
    void addListener(IRequestCompletionListener* listener) {
        listeners_.push_back(listener);
    }

private:
    void notifyCompleted(const Request& r,
        std::chrono::milliseconds wait) {
        for (auto* l : listeners_) l->onCompleted(r, wait);
    }

    std::vector<IRequestCompletionListener*> listeners_;
};
```

Simulation создаёт объект Statistics и регистрирует его как наблюдателя у каждого лифта. После прибытия на этаж лифт вызывает notifyCompleted(), и все подписчики получают уведомление, не подозревая друг о друге.

6.6. Сводная таблица применения паттернов

Паттерн	Категория	Классы / роль в проекте
Singleton	Порождающий	Dispatcher — единая точка диспетчеризации.
Strategy	Поведенческий	IDispatchStrategy + NearestElevatorStrategy / MinQueueStrategy — алгоритм выбора лифта.

Factory Method	Порождающий	StrategyFactory::create — создание стратегии по имени из конфига.
Producer-Consumer	Параллельный	ThreadSafeQueue<Request> между генератором событий и потоками лифтов.
Observer	Поведенческий	IRequestCompletionListener — Statistics подписывается на события лифта.

Перечисленные паттерны охватывают три ключевых аспекта проектирования системы: централизованное управление состоянием (Singleton), гибкость поведения (Strategy + Factory Method), безопасную многопоточную координацию (Producer-Consumer) и слабосвязанные межкомпонентные уведомления (Observer). Совместно они обеспечивают масштабируемость и тестируемость учебного симулятора.